

## BCD encoding bit sizes

James Brakefield

The topic is the number of bits required for various variable length encoding schemes. For ease of comparison will use the examples of a typical address and a typical floating-point value.

Personally, the story begins with the IBM 1620<sup>1</sup> in 1962, almost 63 years ago. As a freshman in college, a sophomore in the same boarding house mentioned his access to a 1620 in the meteorology department. I was quick to join him, learned Fortran<sup>2</sup>, used it in my freshman chemistry class and the next year got a job in the engineering computer laboratory as a “teaching assistant”, also using the 1620. I soon learned the 1620 machine language<sup>3</sup>. This was the era of punch card input.

The 1620 maxed out at 60,000 digits, used a five digit address and a ten digit floating-point value. In binary this would be a 16-bit address and a 32-bit IEEE-754 float. Two other computers that used variable length BCD were Burroughs 2500<sup>4</sup> and IBM 360<sup>5</sup>. Both used a length field in their instructions. For comparison there are six machines:

- A) IBM 1620: 4-bit BCD digit plus flag bit used for both sign and digit string termination.
- B) Alt-1620: hypothetical variable length BCD machine that uses the non-decimal codes (six available) for signs and string termination.
- C) Radix-100: hypothetical variable length radix 100 or radix 128 machine (two digits per byte plus sign/termination flag). It can have a decimal mode with values of 0 to 99 in each byte or a binary mode with values of 0 to 127 in each byte. That leaves one bit per byte for sign/termination.
- D) B 2500            The Burroughs used BCD addresses
- E) IBM 360            The IBM uses binary byte addresses, here I use a six digit BCD
- F) 32-bit mini-computer with 32-bit floating point. The exponent range (+/- 127 equates to a decimal +/- 38 digit range) and the mantissa of 24-bits equates to 7.2 decimal digits. Given that a BCD float loses two or three bits on the vagary of normalization, there is approximate equivalence in both precision and exponent range.

For a modern version a six digit address is minimal and gives the most balanced comparison. The Radix-100 machine has the potential for being an add-on ISA for a 32-bit RISC computer.

---

<sup>1</sup> [https://en.wikipedia.org/wiki/IBM\\_1620](https://en.wikipedia.org/wiki/IBM_1620)

<sup>2</sup> Fortran-IV text book: A guide to fortran IV programming, Daniel D. McCracken, 1965.  
<https://archive.org/details/guidetofortraniv0000dani>

<sup>3</sup> Programming the IBM 1620, 2<sup>nd</sup> ed. , Clarence B. Germain, 1965.

<sup>4</sup> [https://en.wikipedia.org/wiki/Burroughs\\_Medium\\_Systems](https://en.wikipedia.org/wiki/Burroughs_Medium_Systems) two 2-digit length fields

<sup>5</sup> [https://en.wikipedia.org/wiki/IBM\\_System/360](https://en.wikipedia.org/wiki/IBM_System/360) two single hex digit length fields for 2 to 32 digits each  
[https://bitsavers.org/pdf/ibm/360/GC20-1667-1\\_intro360arch.pdf](https://bitsavers.org/pdf/ibm/360/GC20-1667-1_intro360arch.pdf)

First the address comparison:

|             |                        |         |
|-------------|------------------------|---------|
| A) 1620     | 6-digits + flags:      | 30-bits |
| B) Alt1620  | 6-digits + flag digit: | 28-bits |
| C) Radix100 | 3-bytes:               | 24-bits |
| D) B2500    | 6-digits:              | 24-bits |
| E) 360      | 3-bytes:               | 24-bits |
| F) RISC     | 20-bit field:          | 20-bits |

One can say that there is not a big difference in address bit sizes! Base-100 is competitive with B-2500 and IBM-360 representations. And binary is of course more compact than BCD.

The floating-point comparison:

|             |                                   |         |
|-------------|-----------------------------------|---------|
| A) 1620     | 10-digits + flags:                | 50-bits |
| B) Alt1620  | 10-digits + two sign/flag digits: | 48-bits |
| C) Radix100 | 5-bytes:                          | 40-bits |
| D) B2500    | 10-digits:                        | 40-bits |
| E) 360      | 5-bytes:                          | 40-bits |
| F) RISC     | 32-bit float:                     | 32-bits |

The remarks are the same! The use of non-BCD codes for flag and sign digits does not significantly change the bit counts of either addresses or floats (1620 vs. Alt1620)!

Thus any justification for variable length BCD arithmetic is on weak ground. In addition, in my opinion, data byte size typically exceeds the program byte size. This favors the B-2500 or IBM-360 approach of encoding digit string length in the instruction memory rather than in data memory. The Base-100 approach is competitive and could be justified if it is necessary to use long BCD numbers. Performance wise, BCD multiply and divide are not likely to be as efficient as “bignum” binary arithmetic.<sup>6</sup> The “bignum” software packages<sup>7</sup> are not as compact as the “microcode” for variable length hardware.

So, ~60 years later, with balanced hindsight, I find my urge to improve the 1620 relegated to a RISC coprocessor, e.g. Radix-100, of limited utility. Ugh!

My files contain several designs and rationales for both “Alt1620” and “Radix100”. The justification for Alt1620 is as a very simple computer with an intuitive machine language. The justification for Radix100 is that variable length arithmetic removes the need for an equivalent or customized software library and its validation on memory limited 8 and 16-bit microprocessors. A separate issue is that with 32 or 64-bit registers, 8 and 16-digit BCD is possible. This suffices for many applications.

---

<sup>6</sup> [https://en.wikipedia.org/wiki/Arbitrary-precision\\_arithmetic](https://en.wikipedia.org/wiki/Arbitrary-precision_arithmetic)

<sup>7</sup> <https://gmplib.org/> GMP includes many functions besides simple arithmetic. Binary size varies from 300KB to 2MB.

## **04/22/2025 alt-1620 and radix100 implementation choices**

It appears immediately converting BCD addresses to binary preferable as all memory accesses use binary addressing. Marching along BCD strings then can use binary address increment and binary index registers. Implementation can be by a small table of 1..9 multiples for each digit location as address digits are accumulated LSD to MSD.

It is possible to get two flag bits per byte on radix100 memory. Use nine bit wide block RAM and both flag bits are directly available. Conversion between radix-100 and two BCD digits is simple as the LSB is directly available leaving six remaining bits. A six input/seven output table via seven 6-bit LUTs provides the conversion in a single LUT delay. Single digit write to radix-100 memory requires a read/modify/write cycle.

Thus the internal architecture of alt-1620 and radix100 uPs need not be much different. Register file of binary addresses, BCD to binary address conversion circuit, radix-100 to BCD conversion circuit, and various multiply by power of ten circuits. The biggest issue is single digit updates on alt-1620 memory. Of course using 25% more block RAM for single digit writes eliminates the issue.